

Mikrocomputertechnik 1

Einheit 2: RISC-V Basics

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 81

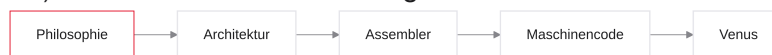
RISC-V Basics

Folie

2 / 81

Agenda für Block 2

1. Architektur-Philosophie — CISC vs. RISC
2. Die RV32I Basis-Architektur — Register File & Spezifikation
3. R-Type-Befehle — Rechnen mit Registern
4. Maschinencode — Assembler → 32-Bit Hex
5. I-Type-Befehle — Konstanten (Immediates)
6. Praxis (Venus) — Simulator und erste Programme



Notizen

Herzlich willkommen zu Block 2 in Mikrocomputertechnik 1. Nach dem Hardware-Fundament — Multiplexer, ALU und Register — wechseln wir die Perspektive: von der Elektrotechnik zur Software. Ziel heute: die Sprache des Prozessors. Wir klären, warum RISC-V so designed ist, lernen Assembler-Grundlagen und schreiben in der zweiten Hälfte erste Programme im Venus-Simulator.

Rückblick: Was wir bisher gebaut haben

- 32-Bit-ALU: Add, Sub, Logik
- Register File: 32 schnelle Speicherplätze
- Multiplexer für steuerbare Datenpfade
- Anbindung an den Arbeitsspeicher (RAM)
- Globales Taktsignal (Clock) synchronisiert alles

Notizen

Kurz rekapituliert: ALU und Register File funktionieren elektronisch, die Clock tickt. Die Maschine ist aber noch „blind“ — niemand sagt, wann welcher Multiplexer umschaltet und ob die ALU addiert oder subtrahiert. Wir brauchen einen Mechanismus, um die Hardware ferngesteuert zu orchestrieren.

Das Problem der Steuerung

- Hardwarekomponenten brauchen Steuersignale
- ALU: z. B. 2-Bit-Code für die Operation
- Register-File-Decoder: 5-Bit-Adresse für das Fach
- Frage: Wie teilen wir der Hardware mit, welche Leitungen wann aktiv sind?
- Antwort: Instruktionen (Befehle) im RAM



Notizen

Die Orchestrierung: Eine 32-Bit-Zahl liegt im RAM. Die Control Unit analysiert sie. Bei Opcode 0110011 weiß sie: addieren — und setzt die Multiplexer. Welche Bit-Kombination was bedeutet, legt der zentrale Vertrag der Informatik fest: die ISA.

1. Die Architektur-Philosophie

Von Software zu Transistoren

- Weg von abstrakter Software zu schaltenden Transistoren
- Hardware und Software brauchen einen festen Vertrag
- Historischer Einfluss teurer Hardware auf Prozessor-Design
- CISC (Intel/AMD): enorme Komplexität in der Hardware
- RISC (ARM/RISC-V): schlanker Gegenentwurf



Notizen

Es gibt nicht „den einen“ Weg, einen Computer zu bauen. Zwei Denkschulen prägen die Schnittstelle Software Hardware. Bevor wir RISC-V-Code schreiben, müssen wir verstehen, in welcher Welt wir uns bewegen und warum unser Befehlssatz anders aussieht als bei einem typischen x86-Laptop.

Was ist eine ISA?

- ISA = Instruction Set Architecture (Befehlssatzarchitektur)
- Scharnier zwischen Software und Hardware
- Definiert, welche Befehle die CPU versteht
- Definiert **nicht**, wie sie elektronisch umgesetzt werden
- Das „Wie“ ist die Mikroarchitektur
- Verbindlicher Vertrag: Compiler-Autor → Chip-Designer

Notizen

Die ISA ist ein Regelwerk: „Sieht der Prozessor ADD, muss er zwei Register addieren.“ Der Compiler (GCC, Clang) darf sich darauf verlassen; der Chip-Designer muss die Transistoren so verdrahten, dass genau das passiert. ISA und Mikroarchitektur sauber trennen — zentral für dieses Modul.

Der historische Kontext (1970er)

- Arbeitsspeicher (RAM) extrem teuer und klein
- Magnetkernspeicher oft nur wenige Kilobyte
- Ziel: Programme so klein wie möglich
- Strategie: Ein Befehl soll extrem viel Arbeit leisten
- Folge: Dominanz der CISC-Architekturen

Notizen

In den 1970ern war RAM sündhaft teuer — oft nur wenige Kilobyte. Ingenieure wollten große Programme vermeiden: Je mehr Arbeit ein einzelner Befehl leistet, desto weniger Bytes braucht das Programm. Das war die Geburtsstunde von CISC.

CISC (Complex Instruction Set Computer)

- Vertreter: Intel x86, AMD64 (viele PCs)
- Philosophie: Programmierer/Compiler entlasten
- Tausende spezialisierte, hochkomplexe Instruktionen
- Ein Befehl kann lesen, rechnen und in den RAM schreiben
- Variable Befehlslänge (z. B. 1 bis 15 Bytes)

```
// Extrembeispiel x86 - String-Operation:  
REP MOVSB  
// Was dieser EINE Befehl tut:  
// 1. Byte aus dem RAM laden  
// 2. an andere RAM-Adresse schreiben  
// 3. zwei Adress-Pointer inkrementieren  
// 4. Zähler dekrementieren  
// 5. wiederholen, bis Zähler = 0
```

Notizen

CISC packt maximale Komplexität in die Hardware. `REP MOVSB` kopiert Datenblöcke, führt Zähler und baut faktisch eine `while`-Schleife — komfortabel für Software, teuer für Silizium.

Die x86-Befehlssatzarchitektur (ISA) ist formal gesehen eine CISC-Architektur. Die Realität moderner x86-Prozessoren ist heute jedoch vielschichtiger: Sie sind ein CISC-RISC-Hybrid.

Um das zu verstehen, muss man streng zwischen der ISA (der Sprache, die der Prozessor versteht) und der Mikroarchitektur (wie der Prozessor intern arbeitet) unterscheiden.

Die "CISC-Fassade" (Die ISA) Aus Sicht des Compilers, des Betriebssystems und des Programmierers ist x86 ein klassisches CISC-Design. Das zeigt sich an folgenden Merkmalen:

Variable Befehlslänge: x86-Befehle können zwischen 1 und 15 Bytes lang sein. (RISC-Architekturen wie ARM haben in der Regel feste Befehlslängen, z.B. immer 4 Bytes).

Komplexe Speicheroperationen: x86 erlaubt Befehle, die Daten direkt aus dem Speicher lesen, mit einem Register addieren und das Ergebnis direkt wieder in den Speicher schreiben. (RISC erfordert dafür separate Load-, Add- und Store-Befehle).

Historischer Ballast: Die Architektur ist seit dem Intel 8086 aus dem Jahr 1978 rückwärtskompatibel. Es gibt Tausende von Befehlen und komplexe Adressierungsmodi.

Der "RISC-Maschinenraum" (Die Mikroarchitektur) Das direkte Ausführen dieser hochkomplexen x86-Befehle in Hardware wurde in den 1990er Jahren zu einem massiven Flaschenhals, der die Taktraten bremste.

Mit dem Intel Pentium Pro (1995) und dem AMD K5 (1996) gab es daher einen radikalen Paradigmenwechsel, der bis heute in Prozessoren wie Intel Core und AMD Ryzen genutzt wird:

Dekodierung: Wenn ein x86-Befehl in den Prozessor geladen wird, führt die CPU diesen nicht direkt aus. Ein Hardware-Decoder zerlegt den komplexen x86-Befehl in kleinere, einfache und gleich lange Teilbefehle. Diese nennt man Micro-Operations (μ ops).

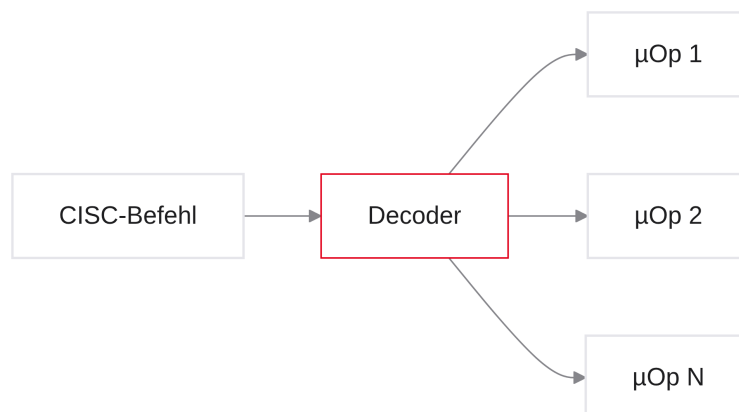
Ausführung: Diese μ ops werden an den eigentlichen Rechenkern der CPU weitergeleitet. Dieser Kern ist eine superskalare, stark optimierte RISC-Engine. Er nutzt typische RISC-Techniken wie Out-of-Order Execution (Befehle werden in der effizientesten Reihenfolge abgearbeitet, nicht zwingend in der programmierten) und Register Renaming.

Folie

10/81

Die Nachteile von CISC

- Gigantische Decoder: lange Befehle — flächen-/stromhungrig
- Unterschiedliche Taktzyklen: Befehl A braucht 1 Takt, Befehl B 120
- Pipelining-Albtraum: unvorhersehbare Arbeitszeiten
- Historischer Ballast: Rückwärtskompatibilität seit den 1970ern
- Ineffiziente Nutzung der Chipfläche



Notizen

Die ALU kann keine Arrays kopieren — nur addieren. CISC-Decoder zerlegen Monster-Befehle in Mikro-Operationen. Pipelines brauchen gleichmäßige Schritte; 1 vs. 120 Takte erzeugen Staus. Die Industrie brauchte einen Neustart.

RISC (Reduced Instruction Set Computer)

- Vertreter: ARM, MIPS, RISC-V
- Philosophie: Hardware simpel, schnell, energieeffizient
- Wenige atomare Befehle (Addieren, Laden, Speichern, ...)
- Keine eingebauten komplexen Schleifenstrukturen
- Kompromiss: Compiler erzeugt mehr Code (mehr Bytes)

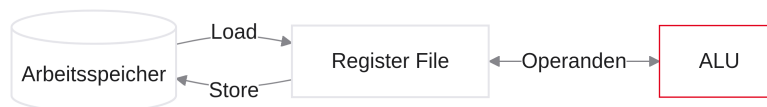
```
// Dieselbe Schleife in RISC:
LOOP:
  Lade Byte aus RAM in Register A
  Schreibe Register A in RAM
  Addiere 1 zu Pointer 1
  Addiere 1 zu Pointer 2
  Subtrahiere 1 vom Zähler
  Springe zu LOOP, wenn Zähler > 0
```

Notizen

In den 1980ern (Berkeley/Stanford): Speicher ist billig geworden — Komplexität aus der Hardware heraus. Der Compiler erzeugt mehr Befehle, die Hardware wird klein und hoch taktfähig. Deshalb steckt in Smartphones RISC, nicht klassisches CISC.

Die eiserne RISC-Regel: Load/Store

- Die ALU kommuniziert **nicht** direkt mit dem RAM
- Berechnungen nur zwischen Registern
- RAM-Zugriff nur über zwei Befehlstypen:
 - Load: RAM → Register
 - Store: Register → RAM



Notizen

Load/Store-Architektur: In CISC dürfen Register und RAM in einem Befehl gemischt werden. In RISC-V ist das verboten — die ALU spricht nur mit dem Register File. Wert aus dem RAM? Erst laden, dann rechnen, dann speichern. Rechen- und Speicherlatenzen bleiben getrennt. Merken Sie sich das für das gesamte Semester.

Die Waage der Komplexität

- Architektur-Design: Nullsummenspiel der Komplexität
- Das Anwendungsproblem bleibt gleich komplex
- CISC: Last auf der Hardware (Transistoren, Strom)
- RISC: Last auf dem Compiler (Software muss clever sein)
- Heute: Mobile/Cloud (Apple, AWS) — RISC gewinnt oft durch Energieeffizienz

Notizen

Die Waage: 100 % Problemkomplexität. CISC drückt auf die Hardware-Seite, RISC auf die Software-Seite. Moderne RISC-Compiler sind Meisterwerke. Weniger Silizium-Komplexität bedeutet bessere Akkulaufzeit und Kühlung — deshalb RISC in praktisch jedem Smartphone.

Warum RISC-V?

- Quelloffener Befehlssatz (UC Berkeley)
- Chip-Bau ohne ISA-Lizenzgebühren
- „Grüne Wiese“ — wenig historischer Ballast
- Modular, klar, didaktisch stark
- Wachsende Relevanz in Embedded Systems und IoT

Notizen

Warum RISC-V an der DHBW — nicht ARM oder x86? Open-Source-ISA: Linux-Ideologie für Hardware. Keine Millionen-Lizenz wie bei ARM. Elegant und logisch, ohne Design-Sünden der 1980er — ideal für die Lehre.

RISC-V in der Industrie

- Keine reine Akademiker-Spielerei mehr
- Western Digital & Seagate: Milliarden Cores in Controllern
- Google: u. a. Steuerlogik in KI-Beschleunigern
- NVIDIA: RISC-V als Steuer-Cores in GPUs
- Relevanz für den Arbeitsmarkt steigt

Notizen

Western Digital, Google, NVIDIA und viele Embedded-Hersteller setzen RISC-V ein. Wer die Architektur beherrscht, versteht einen Chip-Standard der kommenden Jahrzehnte.

2. Die RV32I Basis-Architektur

Überblick RV32I

- Modularer RISC-V-Baukasten
- Fokus: Integer-Basis-Erweiterung RV32I
- Register File als zentrales Arbeitswerkzeug
- 32 Architektur-Register: Hardware- und ABI-Namen
- Application Binary Interface (ABI) als Software-Konvention



Notizen

Wir kennen die RISC-Welt — jetzt den konkreten Vertrag: die ISA von RV32I. Spezifikation, Register-Namen und ABI, damit wir Register im Code korrekt ansprechen.

Die ABI ist der Grund dafür, warum ein Programm, das du für Linux auf ARM (z.B. einen Raspberry Pi) kompilierst, nicht einfach unter Windows auf ARM läuft. Die zugrundeliegende ISA (der ARM-Prozessor) ist exakt dieselbe. Die Hardware würde die Befehle verstehen. Aber die Betriebssysteme haben völlig unterschiedliche ABIs (unterschiedliche Systemaufrufe, unterschiedliche Dateiformate, etc.).

Außerdem ermöglicht eine standardisierte ABI, dass Programme, die in C geschrieben wurden, problemlos Bibliotheken nutzen können, die in C++, Rust oder Go geschrieben wurden. Solange sich die Compiler

Der RISC-V Modulbaukasten

- Nicht ein starrer Satz, sondern modular
- Pflicht: Base Integer (I)
- Optionale Extensions, u. a.:
 - M — Multiply/Divide
 - F / D — Float / Double
 - C — Compressed (16-Bit-Befehle)
- Kursfokus: ausschließlich RV32I

String	Bedeutung
RV32I	32-Bit Basis — Fokus dieses Kurses
RV64GC (= RV64IMAFDC)	64-Bit + G (= IMAFD, inkl. Zicsr/Zifencei) + C

Notizen

Mikrowellen-Controller braucht keine Float-Hardware — Modul weglassen. Jeder Chip braucht nur I (ca. 40 Grundbefehle). G = IMAFD (plus Zicsr/Zifencei); C ist separat — daher RV64GC = RV64IMAFDC. Labor und Klausur: nur der übersichtliche RV32I-Kern.

Hier ist die genaue Aufschlüsselung von RV64IMAFDC:

RV: Steht schlicht für RISC-V.

64: Gibt die Registerbreite an (hier also ein 64-Bit-Prozessor). Es gibt auch RV32 (für kleine Mikrocontroller) oder sogar RV128.

I (Base Integer): Das ist der fundamentale Basis-Befehlssatz für grundlegende Ganzzahl-Operationen (addieren, laden, speichern, springen). Dieses Modul ist als einziges absolute Pflicht.

M (Multiply/Divide): Fügt Befehle für die Hardware-Multiplikation und -Division von Ganzzahlen hinzu (ohne dieses Modul müsste die CPU das mühsam in Software simulieren).

A (Atomic): "Atomare" Befehle. Diese sind zwingend nötig, wenn man Betriebssysteme wie Linux und Multi-Core-Prozessoren nutzen will, damit mehrere Kerne sicher auf denselben Speicherbereich zugreifen können, ohne sich in die Quere zu kommen.

F (Single-Precision Floating-Point): Erlaubt das Rechnen mit einfachen Kommazahlen (32-Bit Gleitkomma).

D (Double-Precision Floating-Point): Erlaubt das Rechnen mit präziseren, großen Kommazahlen (64-Bit Gleitkomma).

C (Compressed Instructions): Ein sehr wichtiges Modul für die Effizienz. Normalerweise sind alle RISC-V-Befehle genau 32 Bit lang. Das "C"-Modul erlaubt es der CPU, die häufigsten Befehle in einem stark komprimierten 16-Bit-Format zu lesen. Das spart massiv Speicherplatz (Code-Größe) und reduziert die Ladezeiten aus dem RAM.

Und was bedeutet nun das "G"? Da die Kombination aus IMAFD (Ganzzahlen, Multiplikation, Atomics, Kommazahlen) der absolute Standard für jeden normalen Allzweck-Computer ist (z.B. für einen Desktop-PC, ein Smartphone oder einen Server, auf dem Linux laufen soll), hat man diese fünf Buchstaben zu

einem einzigen Alias zusammengefasst: G (für General-Purpose).

Daher gilt: IMAFD = G

Wenn man nun noch das C (Compressed) anhängt, wird aus RV64IMAFDC schlicht und ergreifend RV64GC.

RV64GC ist heute der absolute Goldstandard für 64-Bit RISC-V Hardware. Wenn du heute einen RISC-V-fähigen Linux-Rechner kaufst, wird das Betriebssystem voraussetzen, dass die CPU genau diesen "RV64GC"-Befehlssatz beherrscht.

Das "Z" am Anfang der Namen hat eine besondere Bedeutung: Während die großen, grundlegenden Module (wie I, M, A, F, D, C) nur einen einzigen Buchstaben bekommen, steht das "Z" in der RISC-V-Nomenklatur für spezifischere Standard-Erweiterungen, deren Namen aus mehreren Buchstaben bestehen.

Interessant an Zicsr und Zifencei ist ihre Geschichte: Ursprünglich (in älteren Versionen des RISC-V-Standards) waren diese Befehle fest im Basis-Modul "I" (Base Integer) eingebaut. Man hat sie später jedoch herausgelöst, um das Basis-Modul für extrem simple und winzige Mikrocontroller noch kleiner zu machen.

Hier ist, was die beiden Module genau tun:

1. Zicsr (Control and Status Registers) Diese Erweiterung stellt Befehle zur Verfügung, um die internen Verwaltungsregister (CSRs) des Prozessors zu lesen und zu beschreiben.

Was sind CSRs? Neben den normalen Registern (in denen die CPU rechnet, z.B. $5 + 3$), hat eine CPU interne Systemregister. Diese speichern den "Zustand" der Maschine.

Wofür braucht man sie? Über diese CSRs werden grundlegende Systemfunktionen gesteuert:

Interrupts und Ausnahmen: Wenn ein Fehler passiert (z.B. Division durch Null) oder eine externe Hardware (wie eine Tastatur) Aufmerksamkeit braucht.

Privilegienstufen: Um festzulegen, ob der Prozessor gerade im sicheren "Machine Mode" (für die Firmware), "Supervisor Mode" (für das Betriebssystem, z.B. Linux) oder "User Mode" (für deine Apps) läuft.

Timer und Leistungszähler: Um die Uhrzeit des Systems abzufragen oder zu messen, wie viele Taktzyklen vergangen sind.

Fazit: Ein winziger Controller für eine Kaffeemaschine braucht diese Register vielleicht nicht zwingend. Aber jedes System, auf dem ein echtes Betriebssystem laufen soll, muss die Zicsr-Erweiterung haben.

2. Zifencei (Instruction-Fetch Fence) Diese Erweiterung fügt dem Prozessor exakt einen einzigen, aber extrem wichtigen Befehl hinzu: fence.i.

Das Problem: Moderne Prozessoren haben Caches (sehr schnellen Zwischenspeicher). Ein Teil davon (der Instruction Cache) speichert die nächsten Programm-Befehle, die ausgeführt werden sollen. Wenn ein Programm nun aber neuen Code in den Arbeitsspeicher (RAM) schreibt und diesen danach direkt ausführen will, weiß der Instruction Cache der CPU davon nichts. Er würde im schlimmsten Fall veralteten "Müll" ausführen.

Die Lösung: Der fence.i-Befehl ist eine Art Stoppschild und Reset-Knopf für die CPU. Er sagt dem Prozessor im Grunde: "Wirf alle zwischengespeicherten Befehle in deinem Cache weg und lade sie komplett neu aus dem RAM, bevor du weiterarbeitest."

Wofür braucht man das? Das ist absolut essenziell für JIT-Compiler (Just-in-Time). Wenn dein Browser JavaScript ausführt oder dein Rechner Java-Programme laufen lässt, übersetzt das System den Code während der Laufzeit in Maschinencode, schreibt ihn in den RAM und führt ihn sofort aus. Ohne Zifencei würde das nicht sicher funktionieren. Auch Betriebssysteme brauchen den Befehl, wenn sie ein neues Programm in den RAM laden, um es zu starten.

Die Definition von RV32I

- RV: Architektur-Familie RISC-V
- 32: Datenpfad- und Adressbreite 32 Bit
- I: Integer (Zweierkomplement), keine Float-Hardware
- Keine Hardware-Multiplikation/Division in der Basis
- Jede Instruktion exakt 32 Bit (4 Bytes)

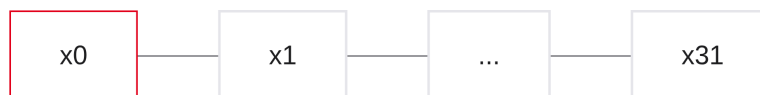
```
Word Size:      32 Bits (4 Bytes)
Registers:      32 Stück
Address Space:  2^32 = 4 Gigabyte
```

Notizen

Spielregeln fürs Semester: 32-Bit-Register, feste 32-Bit-Befehlslänge (kein x86-Längenchaos). „5 × 4“ ohne M-Extension: Schleife aus Additionen — das ist die absolute Basis-Hardware.

Die 32 Architektur-Register

- Register File: exakt 32 Plätze ($x0$ – $x31$)
- Kompromiss: Zugriffsgeschwindigkeit vs. Chipfläche
- Adressierung: 5 Bits ($2^5 = 32$)
- Hardware-Namen: $x0$ bis $x31$
- Ohne Strom: Inhalte sind verloren (volatile)



Notizen

32 Register sind Ihr Arbeitsgedächtnis. Mehr Register → größere Schaltung, oft niedrigere Taktfrequenz. 32 ist der historische Kompromiss; im Befehlswort reichen 5 Bits für die Registerwahl.

Das Spezialregister $x0$ (zero)

- $x0$ physisch oft fest auf 0 verdrahtet (Hardwired Zero)
- Lesen: immer 0
- Schreiben: wird ignoriert (verpufft)
- Nutzen: Operationen aus Standardbefehlen „basteln“ (Kopieren, Negieren, ...)

```
// Metapher:  
const int x0 = 0;
```

Notizen

$x0$ ist die Konstante Null. Schreiben in $x0$ stürzt nicht ab — der Wert bleibt 0. Damit lassen sich Software-Befehle aus bestehenden Gattern ableiten, ohne neue Hardware.

Der Program Counter (PC)

- Zusätzlich zu $x0$ – $x31$: isoliertes PC-Register
- Speichert die Adresse der aktuell auszuführenden / als Nächstes geholten Instruktion
- 32 Bit breit
- Folgeinstruktion: $PC + 4$ (bei RV32I ein Word); Venus zeigt den PC der aktuellen Instruktion



Notizen

Der PC zeigt auf die aktuell auszuführende Instruktion (Venus markiert genau diese). Bei RV32I ist jeder Befehl 4 Bytes — die Folgeinstruktion liegt bei $PC + 4$. Der PC ist der Dirigent des Programmablaufs.

Was ist eine ABI?

- $x0-x31$: physikalische Hardware-Namen
- Software nutzt ABI-Namen (Application Binary Interface)
- Konvention: wofür welche Register dienen
- Hardware erzwingt das nicht — ohne ABI bricht C-Interoperabilität
- Beispiel: $x10 = a0$ (Argument / Rückgabewert)

Hardware	ABI	Nutzung	Preserved?
$x5-x7$	$t0-t2$	temporär	nein
$x8-x9$	$s0-s1$	gespeichert	ja

Notizen

ABI ist ein sozialer Vertrag zwischen Compilern und Programmierern. Die Hardware kennt nur $x5$; die ABI sagt: „ $x5$ heißt $t0$ und ist Schmierzettel.“ So arbeiten Code-Stücke und Bibliotheken zusammen.

ABI: Temporäre Register (t)

- Namen: $t0-t6$ ($x5-x7$, $x28-x31$)
- Zweck: flüchtige Zwischenrechnungen
- Not Preserved (Caller-Saved)
- Nach Funktionsaufruf: Inhalt darf überschrieben sein

```
# Schnelles Zwischenergebnis
add t0, a1, a2    # t0 = a1 + a2
sub a0, t0, a3    # a0 = t0 - a3
# t0 danach nicht mehr benötigt
```

Notizen

t = temporary. Ideal für Zwischensummen. Goldene Regel: nach einem Aufruf (z. B. `print`) dürfen t -Register verloren sein — die Funktion darf sie überschreiben.

ABI: Gespeicherte Register (s)

- Namen: s0–s11 ($x8-x9, x18-x27$)
- Zweck: längerlebige Variablen
- Preserved (Callee-Saved)
- Nutzt die Callee s0, muss sie vorher auf dem Stack sichern und später wiederherstellen

```
int main(void) {  
    int wichtig = 42; // typisch: s-Register  
    fremde_funktion();  
    // wichtig ist danach garantiert noch 42  
}
```

Notizen

s = saved. Tresor für wichtige Werte. Die aufgerufene Funktion muss Ihren s-Wert sichern, wenn sie das Register braucht — kostet Zeit, gibt Sicherheit. Vertiefung: später im Stack-/Calling-Convention-Block.

ABI: Argument-Register (a)

- Namen: a0–a7 ($x10-x17$)
- Zweck: Parameterübergabe an Funktionen
- Rückgabewert: immer in a0
- Ebenfalls flüchtig (Not Preserved)

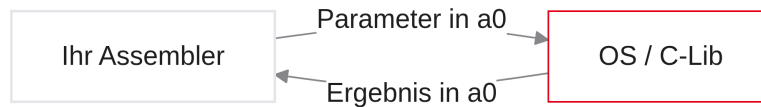
```
int sum(int a, int b) { return a + b; }  
  
// Unter der Haube (ABI):  
// a  → Register a0  
// b  → Register a1  
// Ergebnis → Register a0
```

Notizen

Parameter und Rückgabewerte laufen über a0–a7. `sum(5, 10)`: Compiler legt 5 in a0, 10 in a1; Ergebnis kommt in a0 zurück. Halten wir uns daran, bleibt Assembler professionell und kompatibel.

Warum sich an die ABI halten?

- Venus erzwingt die ABI nicht — rein $\$x\$$ -Code wäre möglich
- Mit C-Bibliotheken / OS-Aufrufen bricht das System sonst
- Beispiel: Print erwartet den Wert typischerweise in `a0`
- Im Kurs: ausschließlich ABI-Namen
- Lesbarkeit und Software-Engineering



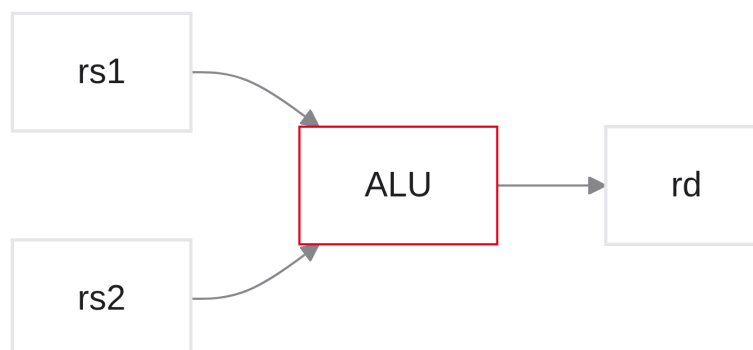
Notizen

Der Prozessor kennt nur $x0-x31$. Aber Syscalls und Bibliotheken suchen Parameter in `a0`. Liegt Ihr Ergebnis in $x31$, kommt Müll auf die Konsole. Deshalb ab heute ABI-Namen.

3. R-Type Befehle (Register)

Einstieg: R-Type

- Einstieg in die Assembler-Programmierung
- Syntax: Ziel, Quelle1, Quelle2
- Arithmetik: Addition und Subtraktion
- Bitweise Logik: AND, OR, XOR
- Shifts: mathematische Bedeutung der Verschiebungen



Notizen

R-Type: „R“ für Register. Zwei Quellregister → ALU → Zielregister. Das ist der Kern der Integer-Arithmetik und -Logik in RV32I — nächster Schritt: konkrete Mnemonics und Encoding.

Folie

30 / 81

Was ist Assembler?

- C, Java oder Python sind Hochsprachen (hardware-unabhängig)
- Prozessoren verarbeiten ausschließlich Maschinencode (0 und 1)
- Assembler: für Menschen lesbare Text-Repräsentation dieses Codes
- Fast 1:1-Mapping: eine Textzeile entspricht einem 32-Bit-Maschinenbefehl
- Mnemonics wie `add` oder `sub` ersetzen unlesbare Binärcodes

```
C-Code:      a = b + c;
Assembler:   add x10, x11, x12
Maschine:    0000000 01100 01011 000 01010 0110011
```

Notizen

Assembler ist ein Wörterbuch: einer 32-Bit-Zahl wird ein lesbares Mnemonic zugeordnet (`add` = addieren). Der Assembler (z. B. in Venus) übersetzt Text in Bits. Da jede Zeile exakt einem Maschinenbefehl entspricht, haben Sie Kontrolle über jeden Taktzyklus.

Folie

31 / 81

Die Grund-Syntax der R-Types

- Grammatik ist strikt — hält den Decoder einfach
- Operationsname (Mnemonic) zuerst
- Operanden strikt durch Kommas getrennt
- Goldene Regel: Zielregister `rd` steht fast immer links
- Danach Quellen: `rs1`, dann `rs2`

```
# Standard-Muster für R-Type:
operation    rd, rs1, rs2

# gelesen als:
# rd wird zu rs1 verknüpft mit rs2
```

Notizen

Bei Register-Rechnungen beginnt die Zeile mit der Operation, dann sofort `rd` (Destination). Das Ziel steht links! Danach die Quellen. Wer dieses Muster kennt, liest 90 % der RISC-V-Befehle blind.

Addition (add)

- Fundamentalster Rechenbefehl der ALU
- $rd = rs1 + rs2$ (32-Bit-Integer, Zweierkomplement)
- Überlauf (Overflow): Hardware schneidet stumm ab — kein Trap
- Dauer: typisch 1 Taktzyklus

```
add t0, t1, t2
# t0 = t1 + t2
```

Notizen

add t0, t1, t2: Control Unit liest t1/t2, ALU auf Addition, Ergebnis nach t0. Sprengt die Summe 32 Bit, wird das 33. Bit verworfen. Overflow-Prüfung ist Software-Sache.

Mapping auf die C-Welt

- Keine Variablennamen wie `temperatur` — nur Register
- Compiler mappt C-Variablen auf ABI-Register (`s0, s1, ...`)
- Keine Typen in der reinen Hardware-Ausführung
- Eine C-Zuweisung wird oft zu einem R-Type
- Ob ASCII oder Zahl: der Programmierer muss es wissen

```
// Hochsprache
int a, b, c; // z. B. a=s0, b=s1, c=s2
a = b + c;

// Generierter Assembler
add s0, s1, s2
```

Notizen

Der Compiler führt Buch: Variable `a` liegt in `s0`. Aus `a = b + c` wird `add s0, s1, s2`. Die ALU addiert Bitmuster — Datentypen existieren auf dieser Ebene praktisch nicht.

Subtraktion (sub)

- $rd = rs1 - rs2$ (Reihenfolge nicht kommutativ!)
- Intern: Zweierkomplement-Trick aus Block 1 (invertieren +1)
- Kein eigener Befehl für „plus negative Konstante“ — später I-Type

```
sub t0, t1, t2
# t0 = t1 - t2    (nicht t2 - t1!)
```

Notizen

sub: erstes Quellregister = Minuend, zweites = Subtrahend. Decoder schaltet den B-MUX auf $\overline{B}$ und Carry-In auf 1 — dieselbe ALU-Hardware wie bei der Addition.

Komplexe Gleichungen

- Maximal zwei Quellen pro Befehl
- C-Zeile $a = b + c - d$ sprengt die Syntax
- ALU hat physisch nur zwei Eingänge
- Unmöglich, drei Operanden in einen 32-Bit-Befehl zu quetschen
- Compiler zerlegt in atomare Schritte

```
// a=s0, b=s1, c=s2, d=s3
a = b + c - d;
```

Notizen

In C sind lange Formeln üblich. In RISC: zwei ALU-Eingänge, zwei Quellfelder. $a = b + c - d$ geht nicht in einem Befehl — der Prozessor braucht Zerlegung.

Lösung: Mehrzeiler (atomare Schritte)

- Zerlegung in Zwischenschritte
- Temporäre Register ($t0-t6$) als Schmierzettel
- Schritt 1: $b + c \rightarrow$ temporär
- Schritt 2: temporär $-d \rightarrow$ Ziel
- Software-Code länger, Hardware bleibt schnell

```
# Ziel: s0 = s1 + s2 - s3
add t0, s1, s2    # t0 = b + c
sub s0, t0, s3    # a = t0 - d
```

Notizen

Erst $b+c$ nach $t0$, dann $t0-d$ nach $s0$. Eine C-Zeile $\rightarrow$ zwei Assemblerzeilen. Genau die RISC-Waage: mehr Code, einfachere Hardware.

Logische Operationen (and, or, xor)

- Bitweise Boolesche Algebra — alle 32 Bits parallel
- and: Bits ausmaskieren ($Y = A \wedge B$)
- or: Bits setzen ($Y = A \vee B$)
- xor: Bits toggeln / Ungleichheit ($Y = A \oplus B$)

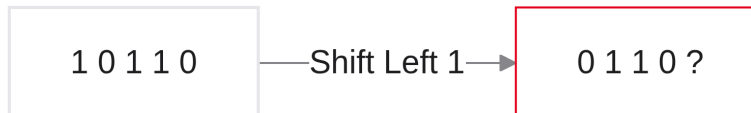
```
# Bit-Maskierung (AND):
# t1 = ...00001111 (0x0F)
# t2 = ...10100101 (0xA5)
and t0, t1, t2
# t0 = ...00000101
```

Notizen

and/or/xor arbeiten bitweise: Bit i mit Bit i . Typisch für Masken (z. B. Farbkanal extrahieren). xor toggelt Bits oder prüft Ungleichheit effizient.

Bit-Shifting (Verschiebungen)

- Bits um N Positionen nach links oder rechts
- Eine Seite: Bits fallen „über die Klippe“
- Andere Seite: Lücken müssen aufgefüllt werden
- In Hardware oft drastisch schneller als Multiplikation



Notizen

Stellen Sie sich 32 Bits wie Perlen auf einer Schnur vor: gemeinsames Verschieben, Randbits gehen verloren, Lücken werden aufgefüllt. Shifts entsprechen oft Multiplikation/Division mit Potenzen von 2 — extrem schnell.

Logical Shift Left (sll)

- Bits nach links; LSB-Lücke mit 0 gefüllt
- Shift um 1 Multiplikation mit 2
- Shift um 3 Multiplikation mit $2^3 = 8$
- Vorsicht: großes N kann das Vorzeichen-Bit am MSB ändern

```
# t1 = 5 (...0101), t2 = 1
sll t0, t1, t2
# t0 = 10 (...1010)
```

Notizen

sll: rechts Nullen nachschieben. Ohne M-Extension sind Shifts Ihr Multiplikations-Werkzeug: $\times 4 = \text{Shift um } 2$.

Logical Shift Right (srl)

- Bits nach rechts; MSB-Lücke mit 0 gefüllt
- Division durch 2 (Integer, Rest verworfen)
- Korrekt nur für nichtnegative Werte
- Bei negativen Zahlen: Null-Fill zerstört das Vorzeichen

```
# t1 = 12 (...1100), t2 = 2
srl t0, t1, t2
# t0 = 3 (...0011)
```

Notizen

srl füllt links mit Nullen. Bei Zweierkomplement-Negativen beginnt links eine 1 — Null-Fill macht die Zahl positiv. Falle!

Arithmetic Shift Right (sra)

- Lösung für negative Zahlen
- Lücke links: Sign-Extension (altes MSB nachfüllen)
- Erhält Zweierkomplement-Mathematik bei Division durch 2

```
# t1 = -4 (1111...1100); Shift-Amount 1 in t2
sra t0, t1, t2
# t0 = -2 (1111...1110)
```

Notizen

sra schaut das MSB an und füllt links mit Einsen bzw. Nullen (Vorzeichenerweiterung). Aus -4 wird -2 — korrektes Zweierkomplement.

Der $x0$ -Trick: Variablen kopieren

- Kein nativer MOV — spart Chipfläche
- Kopie: $t2 = t1 + 0$ über Hardwired Zero
- Compiler „erfindet“ Move aus vorhandener Hardware

```
add t2, t1, x0
# t2 = t1 + 0
```

Notizen

Kein Copy-Befehl nötig: add mit x0 kopiert. Typisch RISC — weniger Mnemonics, gleiche ALU.

Der $x0$ -Trick: Vorzeichen invertieren

- Kein natives NEG
- $0 - X = -X$ über sub und x0

```
sub t1, x0, t1
# t1 = 0 - t1
```

Notizen

sub t1, x0, t1 liefert das Zweierkomplement von t1. Wieder: Reduced Instruction Set durch geschickte Nutzung von x0.

R-Type Übersicht

Alle R-Types: nur Register, treiben primär die ALU; Syntax `befehl rd, rs1, rs2`.

Mnemonic	Name	Operation
<code>add</code>	Add	$rd = rs1 + rs2$
<code>sub</code>	Subtract	$rd = rs1 - rs2$
<code>and</code>	AND	$rd = rs1 \wedge rs2$
<code>or</code>	OR	$rd = rs1 \vee rs2$
<code>xor</code>	XOR	$rd = rs1 \oplus rs2$
<code>sll</code>	Shift Left Logical	$rd = rs1 \ll rs2$
<code>srl</code>	Shift Right Logical	$rd = rs1 \gg rs2$ (0-Fill)
<code>sra</code>	Shift Right Arithmetic	$rd = rs1 \gg rs2$ (Sign-Fill)

Notizen

Werkzeugkiste fürs Labor: ALU liest zwei Register, rechnet, schreibt zurück. Nächster Schritt: Wie wird dieser Text zu Bits auf dem Chip?

4. Maschinencode & Decodierung

Von Text zu Bits

- Text ist nur für Menschen
- Notwendigkeit: 32-Bit-Codierung
- Instruction Format: wo liegt welches Bit?
- Aufbau des 6-Felder-R-Type-Formats
- Wir spielen Assembler — manuell



Notizen

Silizium liest keine Mnemonics — nur Spannungspegel. Der Assembler übersetzt `add t0, t1, t2`

in 32 Bits. Das Format zu verstehen ist Grundlage für den Steuerpfad später in Ripes.

Folie

47/81

Text ist eine Illusion

- Im RAM liegen keine Befehl-Strings
- Fertiges Programm: Hex-Dump (32-Bit-Wörter)
- Decoder muss blind Multiplexer steuern
- Bitmuster muss starr und logisch aufgebaut sein

```
Adresse 0x00: 0x007102B3    # Zahl oder ADD?  
Adresse 0x04: 0x40628333    # SUB?
```

Notizen

Nach dem Kompilieren: Kolonnen von 32-Bit-Zahlen. Der Decoder muss in Nanosekunden entscheiden: ADD? Welche Register? Dafür braucht es eine feste Schablone — das Instruction Format.

Folie

48/81

Das Format-Problem

- Exakt 32 Bits für vier Kerninformationen
- Operation (z. B. Addition)
- Ziel *rd* (5 Bits), Quellen *rs1/rs2* (je 5 Bits)
- Lösung: feste Instruction Formats (Schablonen)

Notizen

Formular mit 32 Kästchen: $3 \times 5 = 15$ Bits allein für Register. Rest für Opcode und Function-Codes. RISC-V definiert mehrere Formate; wir starten mit R-Type.

Das R-Type Instruction Format

Sechs Felder, Bits von rechts (Bit 0, LSB) nach links (Bit 31, MSB):

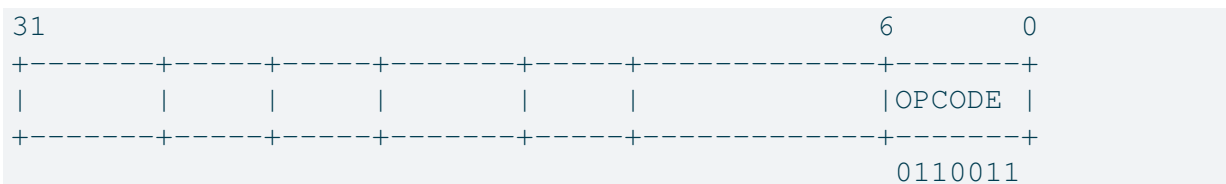
Feld	Bits	Breite	Bedeutung
funct7	31–25	7	Sub-Feature
rs2	24–20	5	Source 2
rs1	19–15	5	Source 1
funct3	14–12	3	Sub-Kategorie
rd	11–7	5	Destination
opcode	6–0	7	Befehlsfamilie

Notizen

Schlüssel fürs Steuerwerk: rechts Opcode, dann rd, funct3, rs1, rs2, links funct7. Die Reihenfolge vereinfacht die Verdrahtung auf dem Die.

Der opcode (7 Bit)

- Bits 0–6 — die Hardware schaut zuerst hier hin
- Definiert die Befehlsfamilie / Struktur
- Beispiel: 0110011 = R-Type-Arithmetik/Logik
- add, sub, and, or, ... teilen sich denselben Opcode

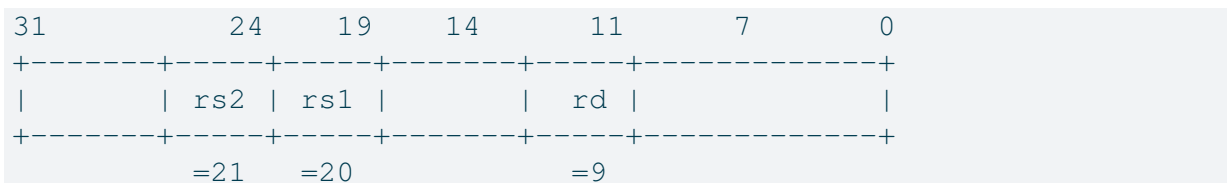


Notizen

Opcode 0110011 → Register-Arithmetik/Logik. Allein reicht das nicht: ADD und SUB brauchen funct3/funct7 zur Unterscheidung.

Die Register-Felder (**rd**, **rs1**, **rs2**)

- Je 5 Bits (Werte 0–31)
- Direkt an Adressports des Register Files
- **rd** → Write-Port *A3*; **rs1/rs2** → Read-Ports *A1/A2*
- Beispiel: *x9* → 01001



Notizen

Reine Adressen: *x9* → 01001. Bits gehen als Kabelbaum an die Ports aus Block 1 — Lesen sofort, Schreiben des ALU-Ergebnisses nach **rd**.

Die Function Codes (**funct3** & **funct7**)

- Gleicher Opcode → Feinunterscheidung über **funct3**/**funct7**
- Gemeinsam: exakte ALU-Operation
- add: **funct3**=000, **funct7**=0000000
- sub: **funct3**=000, **funct7**=0100000
- Erkenntnis: ADD und SUB unterscheiden sich in einem Bit (**funct7**)

Mnemonic	funct7	funct3	opcode
add	0000000	000	0110011
sub	0100000	000	0110011

Notizen

Bit 30 der Instruktion unterscheidet ADD und SUB — der Hebel in der ALU. Parallel-Decode ohne teuren Mikrocode-Pfad.

Live-Decoding (Schritt 1: Mapping)

- Code: `add x9, x20, x21` ($rd = 9, rs1 = 20, rs2 = 21$)
- Opcode R-Type: 0110011
- Register → 5-Bit-Binär
- `add`: `funct3=000, funct7=0000000`

```
rd  (x9)  = 01001
rs1 (x20) = 10100
rs2 (x21) = 10101
```

Notizen

Wir spielen Compiler: Registeradressen und Funct-Werte sammeln — Zutaten für die Schablone.

Live-Decoding (Schritt 2: Assemblierung)

```
[funct7][rs2][rs1][funct3][rd][opcode]
```

```
0000000 10101 10100 000 01001 0110011
```

```
Binär: 0000 0001 0101 1010 0000 0100 1011 0011
```

```
Hex:    0    1    5    A    0    4    B    3
```

```
Maschinencode: 0x015A04B3
```

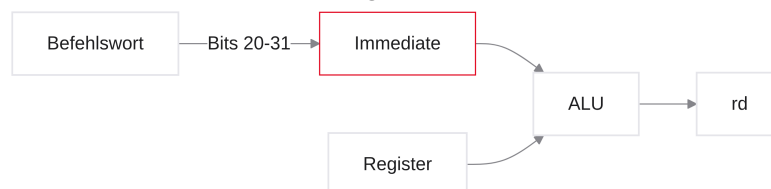
Notizen

Puzzleteile in die Schablone, Nibbles → Hex. Venus schreibt genau diesen Wert bei `add x9, x20, x21` in den Speicher. Ihr erster manuell assemblierter Befehl.

5. I-Type Befehle (Konstanten)

Warum I-Type?

- Limit reiner Register-Verarbeitung
- Konstanten (Immediates) gehören oft zur Instruktion selbst
- I-Type: Felder verschmelzen zu einem Immediate-Feld
- `addi` und die 12-Bit-Grenze
- Pseudo-Instruktionen zur Vereinfachung



Notizen

`a = a + 5`: die 5 liegt nicht im RAM, sondern im Befehlswort. Dafür brauchen wir ein Format mit Immediate — den I-Type.

Konstanten im Code

- Nicht alle Daten liegen dynamisch im Arbeitsspeicher
- Oft brauchen wir feste Zahlen (Konstanten), z. B. Schleifenzähler
- C: `a = a + 5`; — woher kommt die 5?
- Ineffizient: die 5 erst per Load aus dem RAM holen
- Lösung: Konstante wird in den Maschinencode der Instruktion eingebacken

```

int grenze = 100;           // Initialisierung
for (int i = 0; i < 10; i++) // Schleifenzähler
    array[index + 4];       // fester Adress-Offset
  
```

Notizen

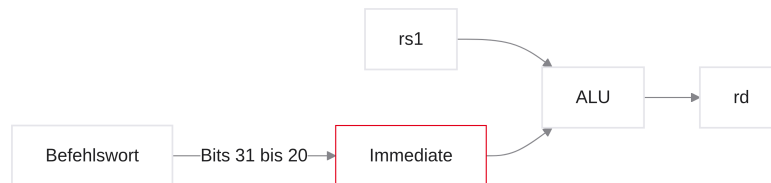
In For-Schleifen ist die 10 eine Konstante. Würde die Hardware jede Konstante erst per Load aus dem RAM holen, wäre das Programm ausgebremst. RISC-V schreibt kleine Konstanten direkt in das

Folie

58/81

Das Immediate (unmittelbarer Wert)

- Beim I-Type belegen die Bits 31 bis 20 das Immediate.
- `rs1` und Immediate sind die beiden ALU-Operanden.
- Das Ergebnis wird nach `rd` geschrieben.



Notizen

Das Immediate ist weder Register noch Speicheradresse, sondern Bestandteil des Maschinenbefehls.

Folie

59/81

Der `addi`-Befehl (Add Immediate)

- Syntax: `addi rd, rs1, immediate`
- Semantik: $rd = rs1 + immediate$
- Typisch für Zähler, Offsets und kleine Konstanten.

```

addi t0, t0, 1
addi s1, x0, 42
addi a0, a0, -4
  
```

Notizen

Das Zielregister steht links; das Immediate ist der dritte Operand.

Das I-Type Instruction Format

Feld	Bits	Breite	Bedeutung
imm[11:0]	31–20	12	vorzeichenbehaftetes Immediate
rs1	19–15	5	erstes Quellregister
funct3	14–12	3	Operation
rd	11–7	5	Zielregister
opcode	6–0	7	Befehlsfamilie

```
| imm[11:0] | rs1 | funct3 | rd | opcode |
```

Notizen

Das I-Type-Immediate ersetzt beim R-Type die Felder für das zweite Quellregister und die obere Funktionskennung.

Die 12-Bit-Grenze

- Das Immediate ist eine vorzeichenbehaftete 12-Bit-Zahl im Zweierkomplement.
- Bereich: -2^{11} bis $2^{11} - 1$, also -2048 bis 2047 .
- Größere Konstanten benötigen mehrere Instruktionen oder `li`.

```
1000 0000 0000 = -2048
0111 1111 1111 =  2047
```

Notizen

Das oberste der zwölf Bits ist das Vorzeichenbit. Daher ist 2048 nicht direkt darstellbar.

Das `li` steht bei RISC-V für "Load Immediate" (zu Deutsch etwa: "Lade einen direkten/unmittelbaren Wert"). Es handelt sich dabei um einen sogenannten Pseudobefehl (Pseudo-Instruction). Das bedeutet: Diesen Befehl gibt es in der eigentlichen Hardware (der ISA) gar nicht. Er ist eine reine Hilfsfunktion des Assemblers (dem Programm, das deinen Text-Code in Maschinencode übersetzt), um dem Programmierer das Leben leichter zu machen.

Das Platzproblem in RISC-Architekturen In einen regulären Befehl passen nur Konstanten (Immediates) mit einer Größe von 12 Bit. Das liegt daran, dass der gesamte Befehl bei RISC-V genau 32 Bit lang ist. Ein Teil dieser 32 Bit wird bereits benötigt, um dem Prozessor mitzuteilen, was er tun soll (Opcode) und welches Register er nutzen soll. Für die eigentliche Zahl bleiben also nur 12 Bit übrig (Zahlen von -2048 bis 2047).

Was passiert, wenn du eine größere Zahl brauchst? Wenn du in deinem Code beispielsweise die Zahl

50.000 in ein Register laden möchtest, passt diese unmöglich in diese 12 Bit. In der Hardware müsstest du das manuell mit zwei echten Befehlen lösen:

lui (Load Upper Immediate): Lädt die oberen Bits der Zahl in das Register.

addi (Add Immediate): Addiert die restlichen unteren Bits dazu.

Weil es für einen Programmierer extrem nervig und fehleranfällig ist, jede größere Zahl manuell in obere und untere Bits zu zerschneiden, gibt es den Pseudobefehl li.

Die Aufgabe von li Als Programmierer schreibst du einfach völlig entspannt: li x1, 50000 (Lade die Zahl 50.000 in das Register x1)

Der Assembler liest das und entscheidet automatisch:

Ist die Zahl zwischen -2048 und 2047? Dann übersetzt er li einfach in einen einzigen, schnellen addi-Befehl.

Ist die Zahl größer? Dann zerlegt er deinen li-Befehl automatisch in eine Kombination aus lui und addi (oder nutzt andere nötige Befehle), um die große Zahl korrekt zusammenzubauen.

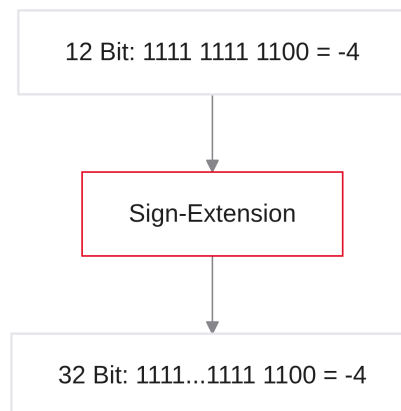
Fazit: li ist ein Komfort-Befehl, der dir die Arbeit abnimmt, dir selbst Gedanken darüber machen zu müssen, ob eine Konstante in 12 Bit passt oder ob du sie über mehrere Instruktionen hinweg stückeln musst.

Folie

62 / 81

Sign-Extension bei Immediates

- Die RV32I-ALU erwartet 32-Bit-Operanden.
- Positive Immediates werden links mit Nullen ergänzt.
- Negative Immediates werden mit dem Vorzeichenbit, also Einsen, ergänzt.



Notizen

Die Sign-Extension bewahrt den Zahlenwert beim Übergang von 12 auf 32 Bit.

Logische I-Types (**andi**, **ori**, **xori**)

- $\text{andi: } rd = rs1 \wedge \text{immediate}$
- $\text{ori: } rd = rs1 \vee \text{immediate}$
- $\text{xori: } rd = rs1 \oplus \text{immediate}$

```
andi t0, t1, 0x0F
ori  t0, t0, 0x80
xori t0, t0, 0x01
```

Notizen

Diese Befehle eignen sich für Bitmasken. Auch logische Immediates werden sign-extended.

Shift I-Types (**slli**, **srli**, **srai**)

- Die Shiftweite `shamt` steht direkt in der Instruktion.
- Bei RV32I sind 0 bis 31 möglich.
- `slli` schiebt links, `srli` logisch rechts, `srai` arithmetisch rechts.

```
slli t0, t1, 2
srli t0, t1, 3
srai t0, t1, 1
```

Notizen

Bei `srai` wird das Vorzeichenbit fortgeführt; bei `srli` werden links Nullen eingefügt.

Warum gibt es keinen **subi**?

- $rs1 - K$ ist identisch zu $rs1 + (-K)$.
- Deshalb genügt `addi` mit einem negativen Immediate.
- Die ISA vermeidet damit eine redundante Instruktion.

```
addi t0, t0, -7    # t0 = t0 - 7
```

Notizen

Ausnahme beim Umformen: Das Gegenstück zu -2048 ist 2048 und passt nicht in das 12-Bit-Feld.

Folie

66/81

Load Immediate (`li`) — Pseudo-Instruktion

- `li rd, wert` ist Assembler-Komfort, kein einzelner RV32I-Befehl.
- Kleine Werte werden meist als `addi rd, x0, wert` umgesetzt.
- Große Werte erfordern eine vom Assembler gewählte Instruktionsfolge.

```
li t0, 123
li t1, 100000
```

Notizen

Pseudo-Instruktionen vereinfachen den Quelltext, erweitern aber nicht die ISA.

Folie

67/81

6. Praxis: Venus Onboarding

Folie

68/81

Vom Text zur Ausführung

- Sie schreiben Assemblertext.
- Venus assembliert ihn zu Maschineninstruktionen.
- Der Simulator macht Register, Speicher und Konsole sichtbar.

Notizen

Sie kennen die Grundbefehle — jetzt Venus. Tippen Sie im Editor, wechseln Sie in den Simulator und beobachten Sie Register und PC Takt für Takt. So wird aus der ISA eine greifbare Ausführung.

Was ist Venus?

- Venus ist ein webbasierter RISC-V-Assembler und -Simulator für die Lehre.
- Er zeigt die Wirkung von Instruktionen auf den Maschinenzustand.
- Venus ist ein Simulator, keine konkrete Mikroarchitektur.

Notizen

Venus emuliert RV32I im Browser: Assembler, Debugger, Registeranzeige. Keine GCC-Toolchain nötig. Wichtig: Venus zeigt ISA-Verhalten, nicht die konkrete Mikroarchitektur eines Chips — den Datenpfad bauen wir später.

Die UI: Der Editor-Tab

- Im Editor schreiben Sie den Assembler-Quelltext.
- Kommentare beginnen mit #.
- „Assemble & Simulate“ prüft und übersetzt den Text.

```
.text
addi t0, x0, 5
```

Notizen

Formatieren Sie in Spalten (Befehl | Operanden | Kommentar). Kommentare mit #. Rote Meldungen genau lesen: fehlende Kommas und `t0` vs. `t o` (Null vs. Buchstabe O) sind die Klassiker.

Basis-Struktur (.text)

- `.text` markiert den Abschnitt mit ausführbaren Instruktionen.
- Labels benennen Adressen und sind später für Sprünge wichtig.

```
.text
main:
    addi t0, x0, 5
    addi t1, t0, 10
```

Notizen

`main`: ist ein Label und kein auszuführender Befehl.

Folie

72/81

Die UI: Der Simulator-Tab

- Nach dem Assemblieren sehen Sie aktuelle Instruktion und Program Counter.
- Register- und Speicheransicht zeigen die Zustandsänderungen.
- Reset stellt den Anfangszustand wieder her.

Notizen

Links sehen Sie Disassembly inkl. Hex der 32-Bit-Instruktion; die farbige Markierung ist der PC (noch nicht ausgeführt). Rechts die Register mit Hardware- und ABI-Namen. Vor jedem Step vorhersagen, welches Register welchen Wert bekommt.

Folie

73/81

Die Register-Ansicht

Anzeige	Bedeutung	Beispiel
<code>x0</code>	fest verdrahtete Null	immer 0
<code>t0</code>	temporäres Register	Zwischenergebnis
<code>a0</code>	Argument- und Rückgaberegister	Dienstcode
<code>pc</code>	Adresse der aktuellen Instruktion (Venus)	Folge: $PC + 4$

- Registerwerte können dezimal, hexadezimal oder binär dargestellt werden.

Notizen

ABI-Namen wie `t0` sind Aliasnamen; die Hardware arbeitet mit Registernummern.

Hex vs. Dezimal

- Dezimal ist für kleine Rechenergebnisse anschaulich.
- Hexadezimal gruppiert Bits in Vierergruppen.
- $0x0000000A$ entspricht dezimal 10.
- $-1 = 0xFFFFFFFF$ bei 32 Bit.

Notizen

Hex ist nur eine andere Schreibweise desselben Bitmusters.

Der Step-Button

- Ein Step führt genau eine Instruktion aus.
- Danach sind Zielregister und PC aktualisiert.
- Schrittweise Ausführung macht falsche Annahmen sichtbar.



Notizen

Nicht blind auf Run: das Programm rast durch. Step = eine ISA-Instruktion (nicht ein Hardware-Takt): Befehl ausführen, Register prüfen, $PC += 4$. Das ist Ihr Standardwerkzeug im Labor.

Trace the Code (mentales Modell)

- Lesen Sie Quellen, Operation und Ziel einer einzelnen Instruktion.
- Notieren Sie Werte vor und nach dem Step.
- Vergleichen Sie Vorhersage, Code und Simulatorzustand.

```
Vorher:  t0 = 5
Code:    addi t0, t0, 1
Nachher: t0 = 6
```

Notizen

Tracing: Zeile lesen → laut vorhersagen („wenn ich klicke, wird $t2 = 15$ “) → Step → Register vergleichen. Der erste abweichende Zustand isoliert den Bug — planloses Steppen lernt nichts.

Labor-Falle 1: Syntax & Kommas

- Die I-Type-Reihenfolge lautet `rd, rs1, immediate`.
- Operanden müssen durch Kommas getrennt sein.
- Assemblerfehler entstehen vor der Simulation, Traps während der Ausführung.

```
addi t0, t1, 5      # korrekt
addi t0 t1, 5      # Komma fehlt
```

Notizen

Syntaxfehler und CPU-Traps sind verschiedene Fehlerklassen.

Labor-Falle 2: Die Immediate-Grenze

- `addi` akzeptiert nur -2048 bis 2047 .
- 2048 ist nicht direkt im I-Type encodierbar.
- Nutzen Sie bei größeren Werten `li`.

```
addi t0, x0, 2047
li t0, 2048
```

Notizen

Die Begrenzung folgt direkt aus den zwölf verfügbaren Bits.

Konsolenausgabe (Intro `ecall`)

- `ecall` übergibt Kontrolle an die Simulationsumgebung.
- In Venus wählen Registerwerte den Dienst und dessen Argumente.
- Für eine ganze Zahl stehen typischerweise Dienstcode in `a0` und Wert in `a1`.

```
li a0, 1
li a1, 42
ecall
```

Notizen

`ecall` ist eine Vorschau auf kontrollierte Übergänge und Traps; es ist kein ALU-Befehl.

Labor-Aufgabe 1: Die erste Gleichung

Ziel: $Y = (A + B) - (C + D)$ mit $A = 5, B = 10, C = 3, D = 2$.

- `li`: Werte in t_0 – t_3 laden
- Zwischensummen in t_4, t_5
- Ergebnis in t_6
- Schrittweise prüfen: $Y = 10$

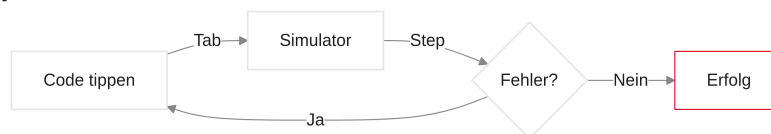
```
.globl main
.text
main:
    # Ihr Code hier...
```

Notizen

Zerlegung: `li` für A–D in t_0 – t_3 , dann $t_4 = A+B, t_5 = C+D, t_6 = t_4-t_5$ (Ergebnis 10). Nach jedem Step prüfen Sie die betroffenen Register. Bei Immediate-Fehlern an die 12-Bit-Grenze denken und ggf. `li` statt großem `addi` nutzen.

Q&A und Hands-On

- Venus im Browser öffnen
- Folien als Spickzettel (R-/I-Type-Übersicht)
- Exzessiv Step + mentales Tracing nutzen
- Support: Syntax-Fehler, die Sie in unter 2 Minuten nicht klären



Notizen

Damit endet der Theorieteil. Folien als Spickzettel offen lassen, Venus starten, formatieren und kommentieren. Step + Tracing nutzen; Frustrationstoleranz hilft. Syntax-Fehler, die Sie in unter zwei Minuten nicht klären, gerne melden. Viel Erfolg bei Ihren ersten RISC-V-Schritten!